Chapter 6: AWS RDS Database Setup

In these last two chapters, you will move the database and Flask app from your local computer to the cloud. On those remote servers, anyone can access your website at any time.

Chapter	Browser	Flask App	Database
Chapters 0-5	local	local	local
Chapter 6	local	local	remote
Chapter 7	local	remote	remote

⚠ **All students are required to go through this process to get a test application to work. Only one team member will deploy the final project.**

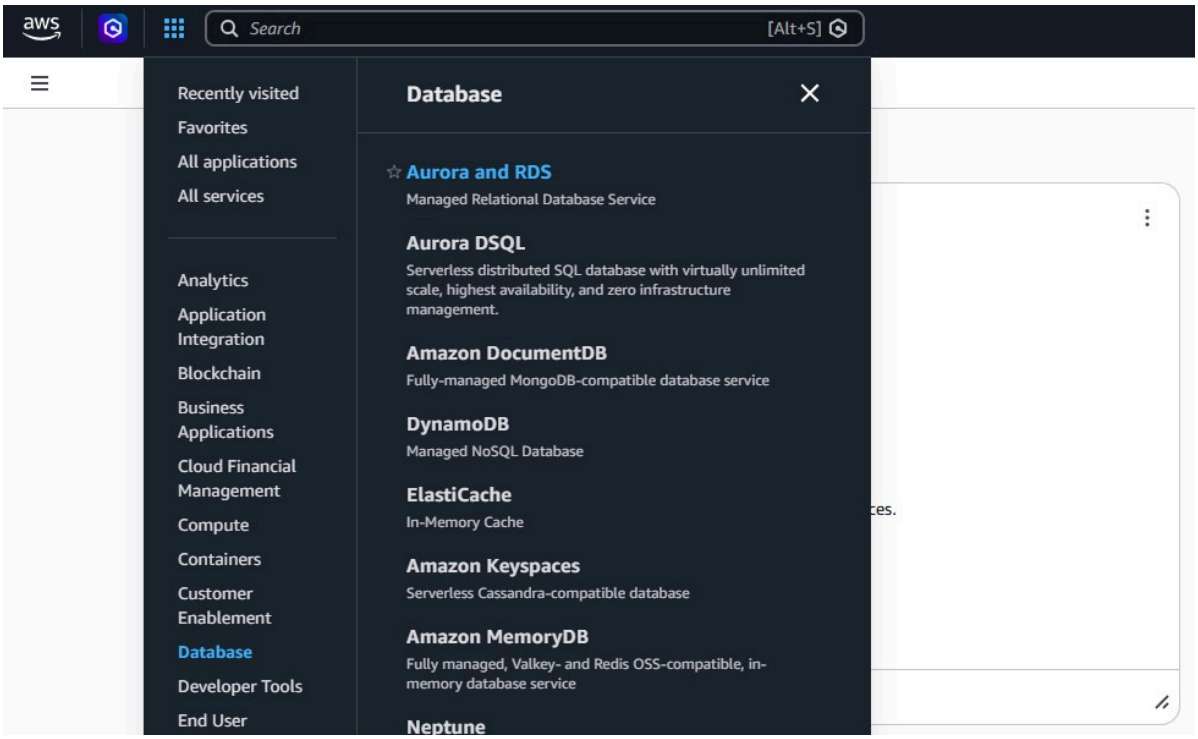
Creating an AWS account

Go to aws.amazon.com and create an account. You will need to put in your credit card details. As long as your select the free tier, your card will not be charged.

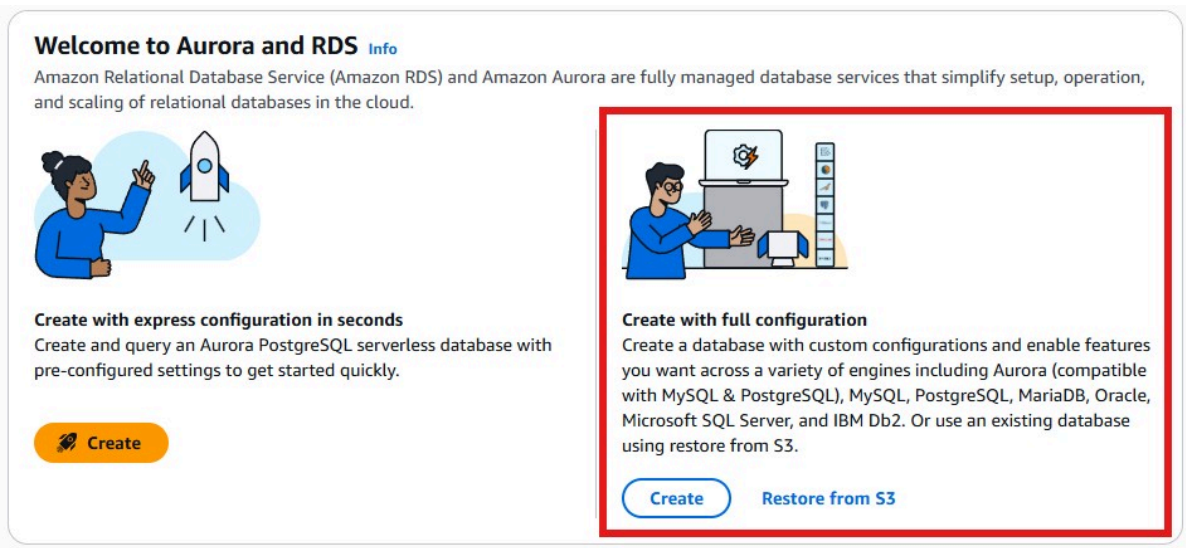
Creating an RDS instance

Amazon RDS (Relational Database Service) provides a managed MySQL database in the cloud.

Log in to the AWS Management Console and find the RDS service either by searching or by navigating to "Database" and then "Aurora and RDS".



Under "Create with full configuration" click "Create"

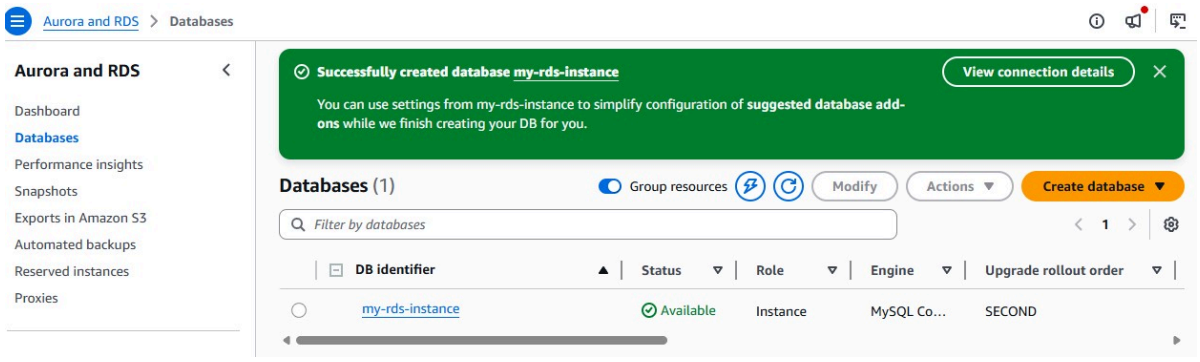


Go through the configuration setup and make selections exactly as follows. If something is not mentioned here you should leave the default value.

- Engine options: MySQL
- Choose a database creation method: Full configuration
- Templates: Free tier
- Deployment options: 1 instance

- Settings
  - Engine version: select the latest
  - DB instance identifier: this is not the database name; It only shows up in AWS. Set it to `my-rds-instance`.
  - Master username: Leave the database manager username as `admin` (instead of root)
  - Credentials management: self-managed
  - Master password: This should usually be a strong password for your database; for this class choose one that you will definitely not forget (like `password`)
- Instance configuration: Set instance type to `db.t4g.micro`
- Storage
  - Storage type: General Purpose SSD (gp2)
  - Allocated storage: 20 GiB
- Connectivity
  - Compute resource: Do **not** connect to an EC2 compute instance. You will do that manually in the next chapter.
  - Public access: **Change to Yes**. This is required to connect your local Flask application to the deployed database.
  - VPC security group: Create a new one. You will adjust its settings in the next step.
  - New VPC security group name: `database-security-group`
  - Open "Additional configuration" and set "Database port" to `3306`
- Open "Additional configuration" at the very bottom of the page and set "Initial database name" to your database name (like `flask_app`)

It might take a couple minutes for AWS to creat the database. You should see the status change from "Creating" to "Available".



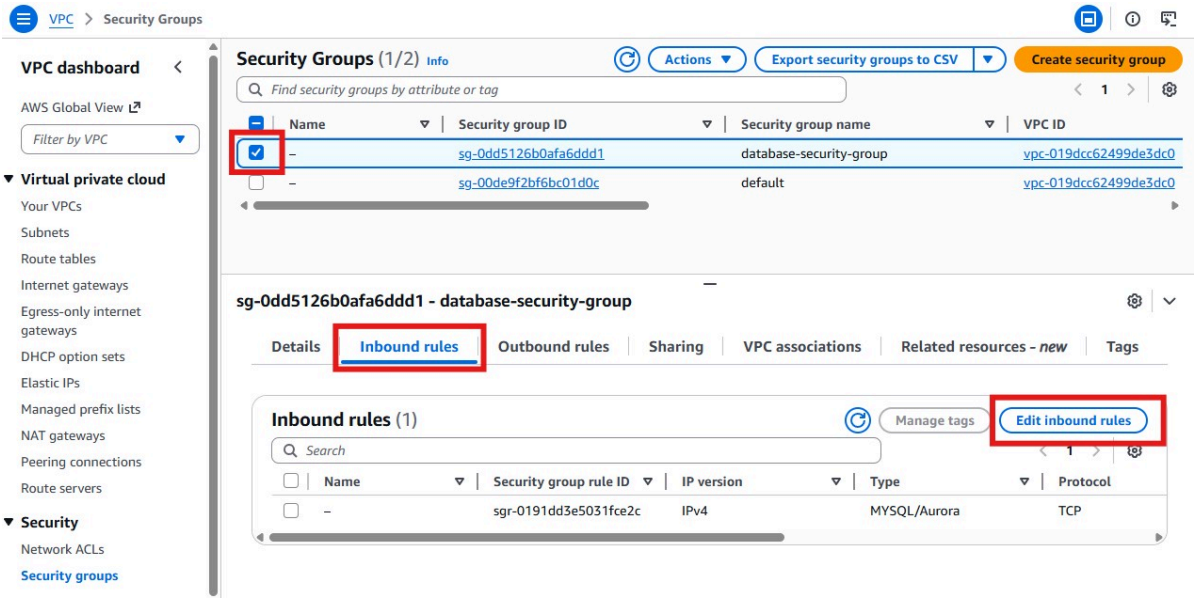
Configuring security groups

Security groups act as a firewall, controlling which IP addresses can access your database through which ports. Follow the steps below to set up the correct settings for your project.

Search "Security Groups" or navigate to "Networking", then click "VPC", and then find "Security groups" on the left side of the new page.

To edit the firewall rules

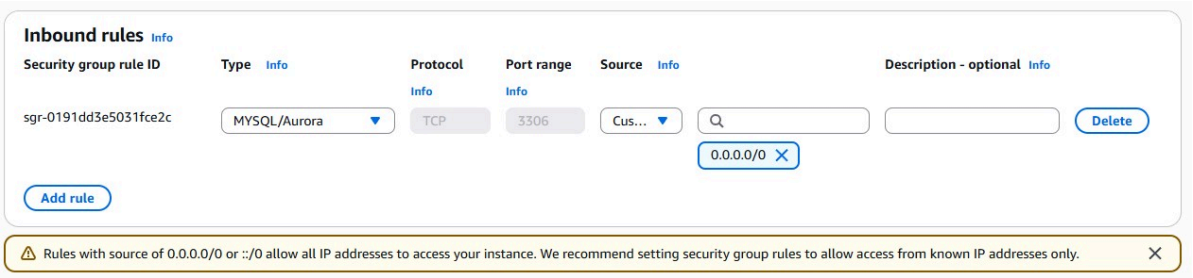
1. Select the security group you created for your database
2. Click "Inbound rules" at the bottom
3. Click "Edit inbound rules"



The database does not need HTTP, HTTPS, or SSH access. The rules should only allow MYSQL/Aurora traffic. Selecting this will automatically set the protocol to TCP and the port to 3306.

The sources are the IP addresses that are allowed to access the database. For class projects in development, open the database to any IP address. Change "Source" to "Anywhere-IPv4" which will set the allowed IP address to `0.0.0.0/0`. This is unsafe and only acceptable during development because it simplifies the setup.

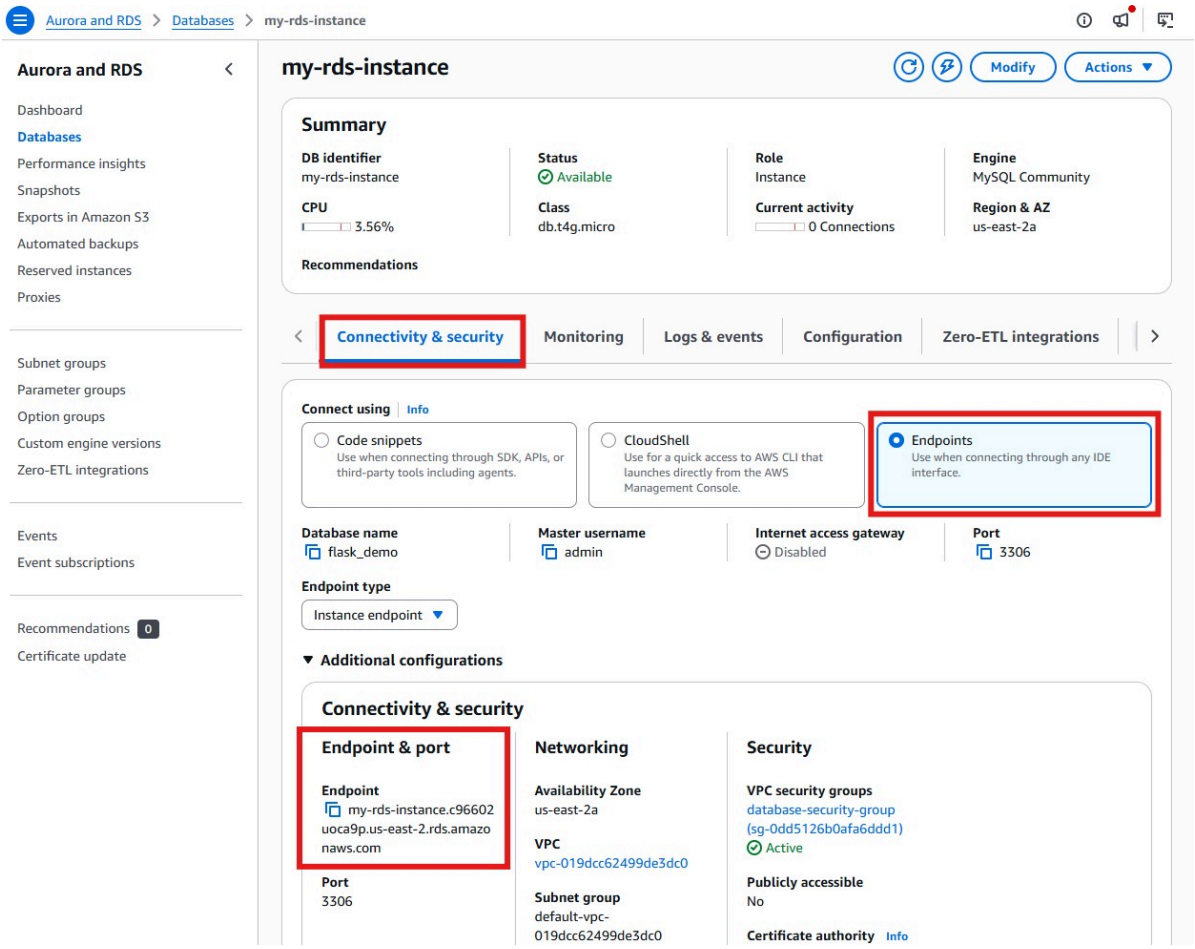
Click "Save rules" at the bottom to apply the changes.



Connecting MySQL Workbench

Retrieve Connection Details

1. In AWS, navigate to the list of your databases (RDS -> Databases)
2. Click on the name of your instance (like `my-rds-instance`)
3. Under "Connectivity" select "Endpoints"
4. Copy the Endpoint (e.g., my-rds-instance.xyz.us-east-1.rds.amazonaws.com)



Adding connection in MySQL Workbench

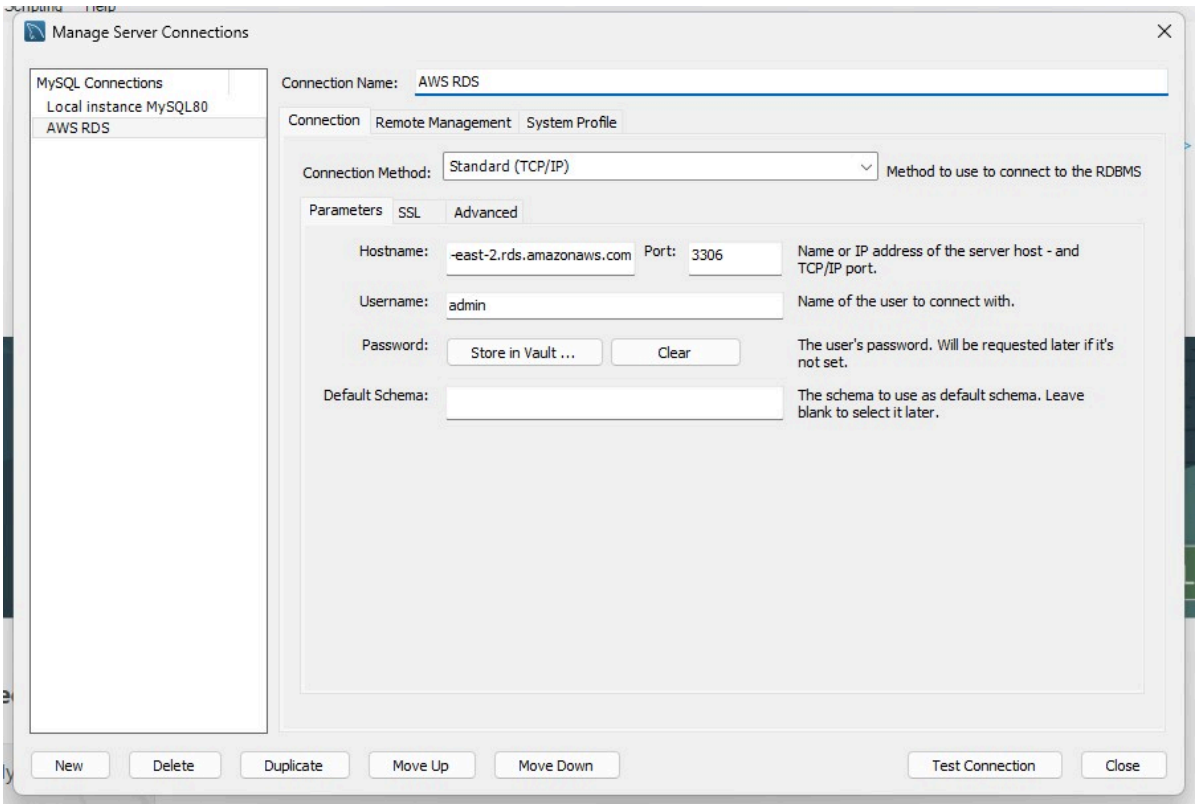
> ⚠ **The obvious path will likely not work; follow these instructions carefully**

MySQL Workbench is not fully compatible with the latest version you selected when setting up RDS. "Add Connection..." will likely not work. After adding the connection through the steps below, you might have to restart Workbench before it shows up at the bottom.

1. Open MySQL Workbench
2. Click "Database", then "Manage Connections..."
3. At the bottom, click "New"
4. Enter the connection details:
    - Connection Name: Something like "AWS RDS"
    - Hostname: Your RDS endpoint

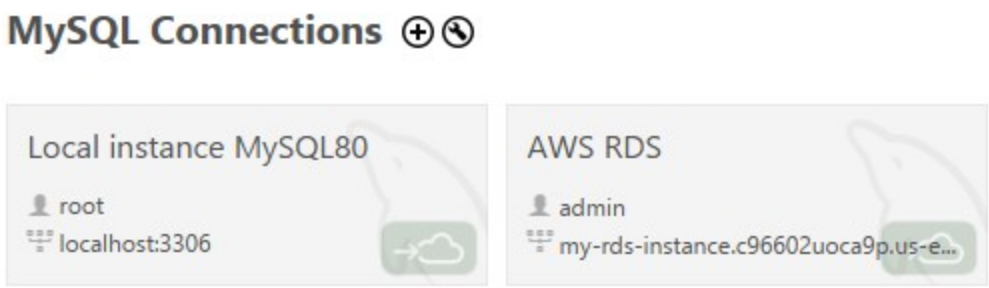
- Port: 3306
- Username: admin (not "root"!)
- Password: Click "Store in Vault..." and enter your RDS password (not AWS)

5. Click "Test Connection" to verify. **You might see a warning that this MySQL version is not supported.** Connect anyway.



> ⚠ **Ignore the warning about unsupported MySQL versions. If you get any other error messages, the most likely source is misconfiguration in the RDS setup. Click 'Modify' in your RDS instance and go through all the steps above to verify you set them up correctly.**

After adding that connection, you should find a new tile at the bottom, next to your local connections. You might have to restart Workbench for it to appear.



Integrating with Flask

Build this minimal Flask application to test the database connection. It works exactly like the connection to the local database. You just need to update the database connection string.

```
mysql+pymysql://username:password@hostname:port/database_name
```

- `mysql+pymysql` remains unchanged
- The username is now `admin`
- Password is now your RDS password, not AWS and not local MySQL
- Hostname is your RDS endpoint that you copied into MySQL workbench in the step before
- Port is `3306`
- `database_name` will be whatever you set as the name when you created the database

Your database connection string should now look something like this:

```
mysql+pymysql://admin:password@my-rds-instance.xyz.us-east-
2.rds.amazonaws.com:3306/flask_app
```

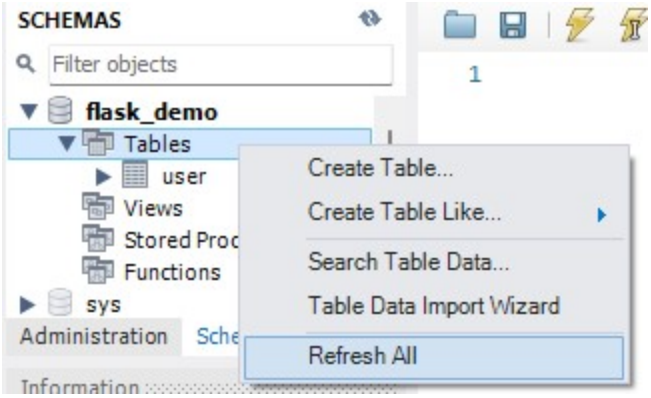
> ⚠ **Putting your RDS password into the code that you put on GitHub is an extreme exception! Never do this outside of this class!**
>
> You should never put passwords into your code, especially if you are putting that code on GitHub. In your public repository, anyone can see that password and use it to access your database. To learn the more complicated but correct and safe way to handle passwords in code, look up "environment variables".

Set up a Flask project with a virtual environment, copy the following code in to `app.py`, install the necessary packages (`flask flask_sqlalchemy pymysql`), and start the app.

app.py

```
from flask import Flask
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = "mysql+pymysql://admin:password@my-rds-instance.xyz.us-east-2.rds.amazonaws.com:3306/flask_app"
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['SECRET_KEY'] = 'your-secret-key-here'

db = SQLAlchemy(app)

class User(db.Model):
   id = db.Column(db.Integer, primary_key=True)
   name = db.Column(db.String(50), nullable=False)

with app.app_context():
    db.create_all()
```

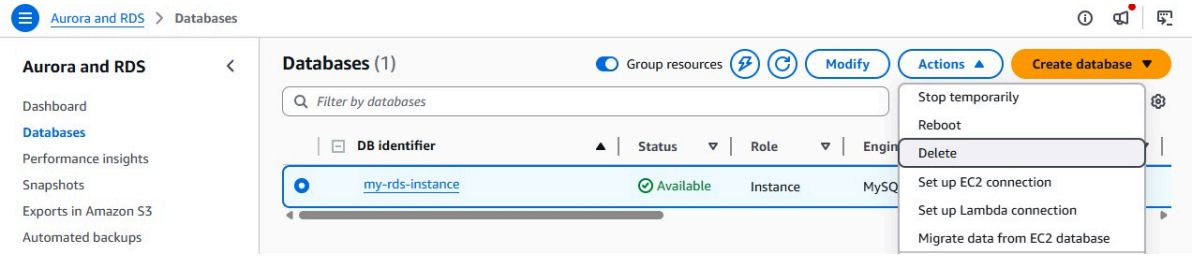
This is a minimal app that does not have an endpoints. It only creates the User table. To verify that this worked, open the database in MySQL Workbench, refresh the tables, and check that the new table appeared.



If you get any error messages, read them carefully. They often tell you exactly what went wrong and how to fix it!

Deleting an RDS instance

Running RDS instances will create costs when your free plan expires. At the top of the databases list, select your database and click "Actions" to see your options.



You can stop an instance temporarily when it is not in use and restart it when you need it. This drastically reduces the running costs.

If you are absolutely sure that you no longer need a database you can delete the instance permanently. This removes all the data you had saved in it and cuts all costs.